

1. Create a dictionary mapping student names to email addresses. Write a program that prompts the user for a name. If the name exists, display the email, otherwise display **contact not found**.

2. Define shopping_list and bought as sets.

```
shopping_list = {"Milk", "Bread", "Eggs"}  
bought = {"Bread", "Eggs"}
```

Compute the set difference to identify unbought items. If items remain, print them if the difference is empty, print Shopping complete

3. Starting with write a program to add a new_student.

```
class_list = ["ram", "sita", "laxman"]
```

The student should only be added if they are not already in the list. Print a confirmation or an already present message.

4. Given a list of votes,

```
votes = ["Blue", "Red", "Blue", "Green", "Blue"]
```

Write a script to count the occurrences of Blue. If the count is 3 or higher, print Blue wins; otherwise, print Blue did not win.

5. Using a dictionary of student grades

```
grades = {"Ram": 92, "Sita": 88}
```

write a program to check if a specific student's name exists as a key. If found, print their grade; otherwise, indicate that the grade is not available.

6. A company accepts an application only if:

```
applicant = { "name": "Priya", "skills": ["Java", "SQL"], "experience_years": 1  
             }  
required_skills = {"Python", "Java"}
```

The candidate knows Python or Java, and has at least 2 years of experience. Check if at least one skill in applicant['skills'] is in required_skills, and experience >= 2. Print **priya qualifies** or **priya does not qualify**

7. Write a Python program that determines whether an airline passenger's cabin baggage is allowed based on two rules: The baggage weight must be 7 kg or less. The item being carried must not be in banned_items.

```
banned_items: {"scissors", "knife", "lighter"}
```

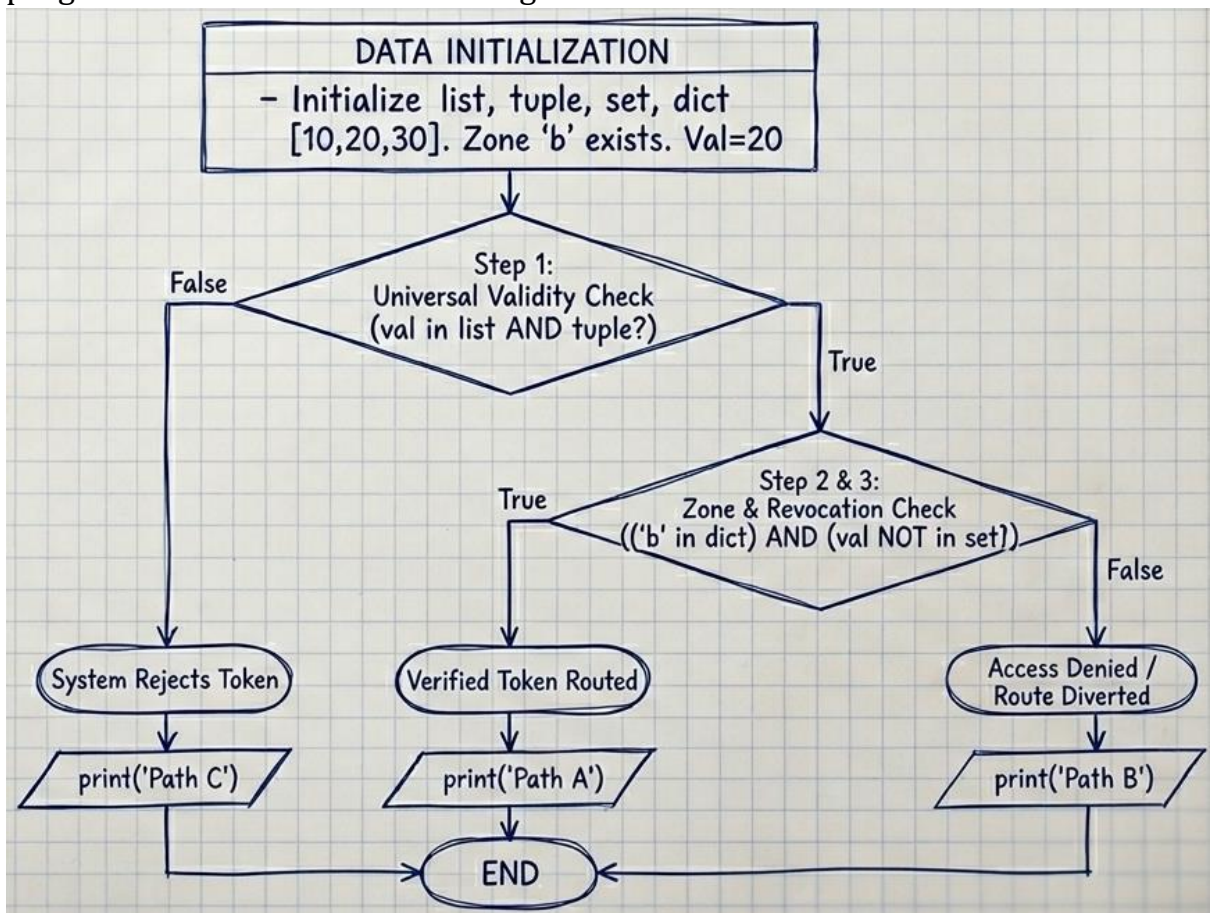
Prompt the user to enter the baggage weight and the name of the item. Convert the item input to lowercase to ensure case-insensitive comparison. If both conditions are satisfied weight <= 7 and item not banned, print **Bag allowed**. Otherwise, print **Bag not allowed**.

8. Write a program to change shyam salary to 8500 in the following dictionary. Given:

```
sample_dict = {
    'emp1': {'name': 'Jhon', 'salary': 7500},
    'emp2': {'name': 'Emma', 'salary': 8000},
    'emp3': {'name': 'Shyam', 'salary': 500}
}
```

9. Store two sets of items for Ram and Laxman. Determine if they have zero items in common. Print **they picked completely different items** or **they have some common items** based on the result.

10. Your task is to write a script that validates an incoming access token and determines the correct network path using three specific checks. The program should follow the exact logic shown in the flowchart.



11. What happens when you initialize a dictionary with duplicate keys, like this:

```
data = {'a': 10, 'b': 20, 'a': 30}
```

The first value 10 is kept and the second 30 is ignored.

The value for a becomes 30.

The dictionary will contain both instances of 'a'.

Python will raise a KeyError during initialization.

12. Which of these cannot be used as a key in a Python dictionary?

10.5

(1,2,3)

[1,2,3]

'key'

13. What is the output of the following?

```
d = {'val': 10}
if d.get('score'):
    print('Found')
else:
    print('Not Found')
```

KeyError

10

Not Found

Found

14. Given items = [10, 10, 20]. What is the result of len (set (items))?

0

1

2

3

15. Which code snippet correctly adds 40 to the existing my_set = {10, 20, 30}?

my_set.append(40)

my_set = my_set + {40}

my_set[3] = 40

my_set.add(40)

16. Create a dictionary menu where **Pizza** is 15, **Burger** is 10, and Salad is 8. Set order = 'Pizza'. Write a program that checks if the order exists as a key in the menu. If it does, print the price of that item; if not, print **item not found**.

17. Initialize a dictionary

```
student_data = {"name": "Sam", "score": 85}
```

Write a program that checks if the **score** is greater than or equal to 80. If it is, add a new key **status** to the dictionary with the value **Pass**. If not, set **status** to **Review**. Print the final dictionary.

18. Define a dictionary

```
database = {"admin": "1234", "user": "abcd"}
```

Define two variables

```
user_input = 'admin'
```

```
user_pass = '1234'
```

Write a conditional that checks if the input_user exists in the database **and** if the password matches the value stored for that user. Print **Login Successful** or **Login Failed**.

19. Initialize a list emails and initialize a set blacklisted emails.

```
emails = ['ram123@gmail.com', 'hari77@gmail.com']
```

```
blacklisted_emails = {'hari77@gmail.com'}
```

Set current_email = 'hari77@test.com'.

Write a program that checks if current_email is in all_emails but not in blacklisted. Print "Email Sent" if safe, or "Blocked" if it fails either condition.

20. Write a script to check if the target key exists in inventory. If it exists, check if the target is not in restricted_zones and the value in inventory is greater than 0.

Print **dispatch item** if all conditions pass. Print **stock error** if it fails the inner check, and **invalid zone** if it fails the outer check.

```
inventory = {'A1': 50, 'B2': 0, 'C3': 10}
```

```
restricted_zones = {'B2', 'Z9'}
```

```
target = 'B2'
```

21. You are developing a student enrollment module. The system must verify course availability and student eligibility using different Python collection types to ensure data integrity. Write a Python script that implements an enrollment gatekeeper using the following requirements.

Create a set called valid_courses containing python, robotics, java and create a list called hs_grades containing integers 9 through 12.

Capture and Store Data, use input() to collect a student's name, course, and grade as an integer. Store these three values inside a single Dictionary named student_records. Use if-else statements to evaluate the data in this exact order:

1. Check if the requested course exists in the valid_courses set. If not, print: {name} selected an invalid course.
2. If the course is valid, check if the student's grade is within the hs_grades list. If the grade is less than 9, print grade too low and if greater than 12, print grade too high.

3. If they pass both checks, apply the robotics rule, if the course is robotics and the grade is 9, they are ineligible.

If they pass, print {name} is approved for {course}

If they fail, print {name} is not eligible for {course} grade too low

